

Contents

Lab Activity-1: Text Representation Methods	1
1. One-Hot Encoding	2
2. Bag-of-Words (BoW)	3
3. N-grams Vectorization (Bigrams)	4
4. TF-IDF Vectorization	5
Lab Activity- 2: Creating word embeddings Using Word2Vec	6
Lab Activity-3: Using Pre-trained Word2Vec Embeddings	8
Lab Project: Sentiment Analysis Using Pretrained Word2Vec	9

Lab Activity-1: Text Representation Methods

How to convert text data into numerical form. we use frequency-based methods to represent text as vectors of numbers.

Text Representation Using Frequency-Based Methods

We will explore four common approaches:

1. **One-Hot Encoding**– Represents each unique word as a binary vector (only one position is 1, others are 0).
Example: “love” → [0, 0, 1, 0, 0]
2. **Bag-of-Words (BoW)** – Represents text by counting how many times each word appears in a sentence or document.
Example: “I love NLP” → [1, 0, 0, 1, 1]
3. **N-grams Vectorization** – Considers the sequence of words by grouping them into pairs or triplets (bigrams, trigrams).
Example: “I love NLP” → “I love”, “love NLP”
4. **TF-IDF (Term Frequency–Inverse Document Frequency)** – Weighs words based on how important they are in a document relative to the entire dataset.
Example: Common words like “is” get lower scores, while unique words like “NLP” get higher scores.

Example Dataset

We use a small dataset:

```
sentences = ["I love NLP", "NLP is fun"]
```

1. One-Hot Encoding

Code

```
sentences = ["I love NLP", "NLP is fun"]

from sklearn.preprocessing import LabelEncoder, OneHotEncoder
import numpy as np

# Step 1: Create vocabulary
words = list(set(" ".join(sentences).split()))

# Step 2: Label Encoding
label_encoder = LabelEncoder()
integer_encoded = label_encoder.fit_transform(words)

# Step 3: One-Hot Encoding
onehot_encoder = OneHotEncoder(sparse_output=False)
integer_encoded = integer_encoded.reshape(len(integer_encoded), 1)
onehot_encoded = onehot_encoder.fit_transform(integer_encoded)

print("Vocabulary:", words)
print("Integer Encoding:", integer_encoded.flatten())
print("One-Hot Encoding:\n", onehot_encoded)
```

Explanation

from sklearn.preprocessing import LabelEncoder, OneHotEncoder
Imports tools to convert words into numerical values.

- LabelEncoder: Assigns an integer to each word.
- OneHotEncoder: Converts integers into binary (0–1) vectors.

```
import numpy as np
```

Loads NumPy for handling numerical arrays.

```
sentences = ["I love NLP", "NLP is fun"]
```

Our example dataset.

```
words = list(set(" ".join(sentences).split()))
```

- Joins sentences and splits them into words.
- set() removes duplicates.
- Example: ['I', 'fun', 'love', 'is', 'NLP']

```
label_encoder.fit_transform(words)
```

- Converts each unique word into an integer label.

- Example: [0, 1, 2, 3, 4]

```
integer_encoded.reshape(len(integer_encoded), 1)
```

- Reshapes data into a column, required by the OneHotEncoder.

```
onehot_encoder.fit_transform(integer_encoded)
```

- Converts integer codes into binary vectors (one 1 per word).

Output Table

Word	One-Hot Vector
I	[1, 0, 0, 0, 0]
fun	[0, 1, 0, 0, 0]
love	[0, 0, 1, 0, 0]
is	[0, 0, 0, 1, 0]
NLP	[0, 0, 0, 0, 1]

2. Bag-of-Words (BoW)

Code

```
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(sentences)

print("Vocabulary:", vectorizer.get_feature_names_out())
print("BoW Matrix:\n", X.toarray())
```

Explanation

```
from sklearn.feature_extraction.text import CountVectorizer
```

Imports a tool that counts how many times each word appears in each document.

```
vectorizer = CountVectorizer()
```

Creates a Bag-of-Words converter.

```
vectorizer.fit_transform(sentences)
```

- Learns the vocabulary.
- Converts each sentence into a vector of word counts.

```
get_feature_names_out()
```

- Lists all unique words in the vocabulary.

```
X.toarray()
```

- Converts the count matrix to a readable form.

Output Table

Sentence	I	fun	is	love	NLP
I love NLP	1	0	0	1	1
NLP is fun	0	1	1	0	1

3. N-grams Vectorization (Bigrams)

Code

```
vectorizer = CountVectorizer(ngram_range=(2, 2))
X = vectorizer.fit_transform(sentences)

print("Bigrams Vocabulary:", vectorizer.get_feature_names_out())
print("Bigrams Matrix:\n", X.toarray())
```

Explanation

```
ngram_range=(2, 2)
```

- Extracts *bigrams* (two-word combinations).
- Example:
 - “I love NLP” → “I love”, “love NLP”
 - “NLP is fun” → “NLP is”, “is fun”

```
fit_transform(sentences)
```

- Counts how many times each bigram appears.

Output Table

Sentence	i love	love nlp	nlp is	is fun
I love NLP	1	1	0	0
NLP is fun	0	0	1	1

4. TF-IDF Vectorization

Code

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(sentences)

print("Vocabulary:", vectorizer.get_feature_names_out())
print("TF-IDF Matrix:\n", X.toarray())
```

Explanation

`from sklearn.feature_extraction.text import TfidfVectorizer`
Imports the TF-IDF (Term Frequency–Inverse Document Frequency) converter.

What it does:

- Gives higher importance to words that are frequent in one document but rare across others.

`fit_transform(sentences)`

- Calculates TF-IDF for each word.

`get_feature_names_out()`

- Displays all unique words.

`X.toarray()`

- Converts the TF-IDF matrix to a dense, readable array.

Output Table (approximate values)

Sentence	fun	i	is	love	nlp
I love NLP	0.00	0.58	0.00	0.58	0.58
NLP is fun	0.58	0.00	0.58	0.00	0.58

Lab Activity- 2: Creating Word Embeddings Using Word2Vec

How to create word embeddings using the Word2Vec algorithm.

Step 1: Prepare Your Text

Code:

```
# Your text
my_text = "I love NLP and I want to learn Word2Vec embeddings"

# Tokenize text into words
sentences = [my_text.lower().split()] # Lowercase and split
print("Tokenized sentence:", sentences)
```

Explanation:

- `my_text.lower().split()`
 - Converts all text to lowercase to avoid duplicates like “NLP” and “nlp”.
 - Splits the sentence into individual words (tokens).
 - Example output: `[['i', 'love', 'nlp', 'and', 'i', 'want', 'to', 'learn', 'word2vec', 'embeddings']]`

Step 2: Train Word2Vec Model

Code:

```
!pip install gensim
from gensim.models import Word2Vec

# Train Word2Vec model
model = Word2Vec(sentences, vector_size=10, window=2, min_count=1,
workers=2)
```

Explanation:

- `Word2Vec(sentences, ...)`
 - `sentences`: Input tokenized sentences.
 - `vector_size=10`: Each word will be represented as a 10-dimensional vector.
 - `window=2`: Context window for neighboring words.
 - `min_count=1`: Include all words appearing at least once.
 - `workers=2`: Number of CPU threads for faster training.

What it does:

- The model learns numerical embeddings for each word, capturing semantic relationships.

Step 3: Extract Word Embeddings

Code:

```
# Create embedding table
embedding_table = {}
for word in model.wv.index_to_key:
    embedding_table[word] = model.wv[word]

# Print embedding table
for word, vector in embedding_table.items():
    print(f"{word}: {vector}")
```

Explanation:

- `model.wv.index_to_key` → List of all words in the vocabulary.
- `model.wv[word]` → Retrieves the 10-dimensional vector for each word.
- `embedding_table` → Dictionary storing each word and its vector.
- Output can be used for further NLP tasks (similarity, clustering, etc.).

Step 4: Example Output Table

Word	Vector (10-dimensional)
i	[0.012, -0.034, ...]
love	[-0.021, 0.045, ...]
nlp	[0.002, -0.010, ...]
and	[-0.007, 0.028, ...]
want	[0.011, -0.019, ...]
to	[-0.004, 0.033, ...]
learn	[0.009, 0.022, ...]
word2vec	[-0.012, 0.018, ...]
embeddings	[0.014, -0.021, ...]

Each row represents a word and its vector. this is your embedding table.

Lab Activity-3: Using Pre-trained Word2Vec Embeddings

CODE

```
import gensim.downloader as api

# Load the 'word2vec-google-news-300' model
model = api.load("word2vec-google-news-300")

# Retrieve the vector for the word 'dog'
vector = model['dog']
print(vector)

# Find words most similar to 'dog'
similar_words = model.most_similar('dog', topn=5)
print("Words most similar to 'dog':", similar_words)
```

Explanation

1. `api.load("word2vec-google-news-300")`
 - Downloads (in Colab only) and loads the pre-trained Word2Vec model with 300-dimensional vectors.
 - Does not download anything to your local laptop.
2. `model['dog']`
 - Retrieves the 300-dimensional embedding for the word "dog".
3. `model.most_similar('dog', topn=5)`
 - Finds the 5 words whose embeddings are closest to "dog" in vector space.

Example Output

Vector for 'dog':

[0.123, -0.042, 0.051, ..., 0.082] # 300-dimensional

Vector length: 300

Words most similar to 'dog': [('puppy', 0.88), ('dogs', 0.87), ('cat', 0.84), ('pet', 0.82), ('hound', 0.81)]

Lab Project: Sentiment Analysis Using Pretrained Word2Vec

Step 1: Import Required Libraries

```
import gensim.downloader as api
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

Explanation:

- `gensim.downloader as api`: lets us load pretrained Word2Vec embeddings easily.
- `numpy`: for working with numeric arrays.
- `Sequential` and `Dense`: to build an Artificial Neural Network (ANN).
- `Tokenizer` and `pad_sequences`: to convert sentences into lists of tokens (numbers).

Step 2: Load Pretrained Word2Vec Model

```
# Import the downloader API
import gensim.downloader as api
model_w2v = api.load("word2vec-google-news-300")
print("Model loaded successfully!")
```

Explanation:

- Loads the **Google News Word2Vec model** (each word represented as a 300-dimensional vector).
- This pretrained model already knows relationships between words — no need to train from scratch.

Step 3: Define Example Sentences and Labels

```
# Positive = 1, Negative = 0
sentences = [
    "I love this product",          # positive
    "This movie was amazing",      # positive
    "I feel happy today",          # positive
    "What a great experience",     # positive
    "I am very satisfied",          # positive
    "This is absolutely wonderful", # positive
```

```

    "I enjoyed every moment",      # positive
    "I hate this place",          # negative
    "This is the worst thing ever", # negative
    "I am very disappointed"      # negative
]

labels = [1,1,1,1,1,1,1,0,0,0] # 1 = positive, 0 = negative

```

Explanation:

- 10 example sentences: 7 positive and 3 negative.
- `labels` tells the model which sentiment each sentence has.

Step 4: Split Data into Training and Testing Sets

```

X_train = sentences[:7]
y_train = labels[:7]

X_test = sentences[7:]
y_test = labels[7:]

```

Explanation:

- 7 sentences for **training**, 3 sentences for **testing**.

Step 5: Convert Sentences to Vectors Using Word2Vec

```

def sentence_vector(sentence, model):
    words = sentence.lower().split()
    word_vectors = [model[w] for w in words if w in model]
    if len(word_vectors) == 0:
        return np.zeros(300)
    return np.mean(word_vectors, axis=0)

X_train_vec = np.array([sentence_vector(sent, model_w2v) for sent in
X_train])
X_test_vec = np.array([sentence_vector(sent, model_w2v) for sent in
X_test])

```

Explanation:

- For each sentence:

- Split into words.
 - Retrieve each word's embedding from Word2Vec.
 - Average all word embeddings → 1 vector per sentence (300-dimensional).
- If a word isn't found in the model, it's ignored.

Step 6: Build the ANN Model

```
model = Sequential([
    Dense(128, activation='relu', input_shape=(300,)),
    Dense(64, activation='relu'),
    Dense(1, activation='sigmoid')
])
```

Explanation:

- Input layer: 300 features (from Word2Vec).
- Two hidden layers:
 - 128 neurons (ReLU activation)
 - 64 neurons (ReLU activation)
- Output layer: 1 neuron (sigmoid → outputs value between 0 and 1 for sentiment).

Step 7: Compile and Train the Model

```
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
model.fit(X_train_vec, np.array(y_train), epochs=30, verbose=1)
```

Explanation:

- `adam`: adaptive learning optimizer.
- `binary_crossentropy`: used for binary classification (positive vs. negative).
- Train for 30 epochs (full passes over the training data).

Step 8: Evaluate on Test Data

```
loss, accuracy = model.evaluate(X_test_vec, np.array(y_test))
print(f"Test Accuracy: {accuracy*100:.2f}%")
```

Explanation:

- Evaluates how well the model performs on unseen sentences.
- Prints out the test accuracy.

Step 9: Test Prediction Examples

```
test_sentences = ["I really love this!", "This was terrible", "What an
awesome experience"]
test_vecs = np.array([sentence_vector(s, model_w2v) for s in
test_sentences])

predictions = model.predict(test_vecs)

for sent, pred in zip(test_sentences, predictions):
    sentiment = "Positive" if pred > 0.5 else "Negative"
    print(f"{sent} → {sentiment} ({pred[0]:.2f})")
```

Explanation:

- Creates **new unseen sentences**.
- Converts them to vectors using the same function.
- The model predicts a score between 0 and 1:
 - $> 0.5 \rightarrow$ **Positive**
 - $\leq 0.5 \rightarrow$ **Negative**
- Uses the Python built-in function `zip()` to pair each sentence with its prediction.

Step 10: Example Output

I really love this! → Positive (0.93)

This was terrible → Negative (0.12)

What an awesome experience → Positive (0.91)